Contents

1 Implementation Report – fault-triangle quality remesh

Plan: `meshing/docs/PLAN_fault_triangle_quality_2026-05-26.md` (Phases 1–3 C) **Date:** 2026-05-26 **Scope:** lift fault-triangle quality of `meshing/results/msh/safs_fault_box_nwcut_500m_lcfar3000_z0embed` to min-angle $\geq 25^\circ$ / tri_q ≥ 0.5–0.6, **without modifying any existing code or mesh**. All artefacts live in `experimental_mesh_refinement/`.

1.1 Result – TARGET MET

Metric	Baseline mesh	`_triq` (CGAL)	Target	Pass
Bulk Q1 `tet_emin` [m]	112.5	**113.45**	≥ 100	✓
Bulk Q2 `η_min`	0.184	**0.2032**	> 0.1	✓
Fault `tri_qmin`	0.2446	**0.6647**	≥ 0.5 (0.6 stretch)	✓
Fault worst min-angle	9.10°	**26.59°**	$\geq 25^\circ$ (30° stretch)	✓
Fault tris `tri_q < 0.5`	45	**0**	0	✓
Fault tris min-angle $< 25^\circ$	187	**0**	0	✓
`mesh_zmax`	0.0	**0.0**	0 exactly	✓
Fault embedded (2-tet faces)	yes	**60658 / 60658**	all	✓
Connected fault surfaces	1	**1**	1	✓
Sampled Hausdorff (STL↔input)	—	**0.42 m**	≤ 25	✓
n_tet / n_fault_tri	3.69M / 42358	1.16M / 60658	—	—

The 5 worst fault triangles are now all $\geq 26.6^\circ$ and at depth (z = −143 … −11710 m); the shallow z=0 trace needles (the 2026-05-20 blow-up seed) are gone. `pytest test_fault_triangle_quality.py` → 2 passed (new mesh clears the gate; baseline fails it, confirming the gate has teeth).

1.2 What was implemented

1. **`remesh_fault_stl.py`** — STL surface-remesh driver. Reads the cut fault STL, remeshes the surface holding the z=0 trace + borders, snaps the top boundary to exactly z=0, writes a new STL. Engines:
   - `--engine cgal` (default) — calls the compiled CGAL tool; **clears the full target**.
   - `--engine mmgs` — MMG `mmgs_O3`, pins the z=0 trace as Medit `RequiredEdges`; gets

9.1°→14.4° but cannot collapse the pinned trace segments (see below).
- `--engine gmsh|pymeshlab` — `NotImplementedError` (Phase-3 A/B, not needed).
- Reports a surface-quality gate, sampled Hausdorff, connected-component and z=0-trace-segment checks.

2. **`remesh_cgal/remesh_fault_cgal.cpp` + `CMakeLists.txt`** — ~90-line CGAL tool: `isotropic_remeshing` with every open-boundary edge constrained, `protect_constraints=false` + `collapse_constraints=true` + `relax_constraints=true`. This collapses the short z=0 clip segments while keeping the trace a planar 1-D feature. OFF in/out (double precision) to avoid STL float drift. Built in the `cgal-61` env.
3. **`test_fault_triangle_quality.py`** — pytest gate on the embedded fault (worst min-angle ≥ 25, tri_qmin ≥ 0.5); skip-on-missing-mesh; also asserts the baseline FAILS the gate.
4. Outputs: `SAFS-…-ALT6_500m_clean_clip_nwcut_triq.stl` (remeshed STL), `safs_fault_box_nwcut_500m_lc` (new mesh).

1.3 Why CGAL (the mmgs → CGAL decision)

mmgs improved 187→7 sub-25° tris (9.1°→14.4°) but **structurally cannot reach 25°**: pinning the z=0 trace as `RequiredEdges` keeps it planar but those edges are non-collapsible, so the short clip segments stay short and force ~7 trace-adjacent needles **and** a bulk Q1 regression (sub-100 m fault edges → tet_emin 73.6 m). The user chose the CGAL fallback (Phase 3 C). CGAL's `collapse_constraints` collapses the short trace segments while `relax_constraints` slides survivors along the (planar) polyline — the exact capability mmgs lacks.

CGAL parameter sweep (target = fraction × median edge ≈ 500 m)

target [m]	iters	verts	tris	worst angle	tri_qmin	#<25°	fault_emin
500	5	21719	42184	9.69°	0.263	10	176
400 (0.8×med)	**8**	**31274**	**61206**	**25.7–26.6°**	**0.61–0.66**	**0**	**179**
300	8	84795	167130	25.21°	0.602	0	151
250	10	85642	168785	23.33°	0.593	4	128
200	10	106833	210980	21.45°	0.536	7	88

`target = 0.8 × median, iters = 8` is the sweet spot: clears the target with a modest 1.43× tri-count increase and keeps `fault_emin > 100` (so bulk Q1 passes). target ≈ median is a near-no-op; smaller targets over-refine and re-seed boundary slivers (and drop fault_emin < 100 → Q1 fail). This is the locked default (`DEFAULT_CGAL_TARGET_FRACTION = 0.8`, `DEFAULT_CGAL_ITERS = 8`).

1.4 Decisions / deviations from the plan

- **Medit RequiredEdges writer (mmgs path).** Plan step 3 wrote the `.mesh` with meshio, but meshio cannot emit `RequiredEdges/RequiredVertices`, which are essential to pin the z=0 trace (a free mmgs run lifts the trace up to ~45 m off-plane, with no clean z-gap to snap back). Replaced the medit *write* with an explicit writer (`write_medit`); STL is still *read* with meshio.

- **Explicit Medit reader for mmgs output.** Plan step 5 read the remeshed `.mesh` with meshio, but meshio's medit reader rejects mmgs's `RequiredEdges/Ridges/Normals/Tangents` sections. Added `_read_medit_tris` (Vertices+Triangles only) and use it for mmgs output.
- **Default engine = cgal** (plan led with mmgs as primary). mmgs is retained behind `--engine mmgs`; cgal is default because it is the only engine that meets the target — consistent with the plan's Phase-3 escalation, chosen by the user.
- **CGAL writes/reads OFF, not STL** (per the plan's Phase-3 C note on STL float drift); the Python wrapper converts STL→OFF→STL and snaps z=0.

1.5 Files

- [created] `experimental_mesh_refinement/remesh_fault_stl.py`
- [created] `experimental_mesh_refinement/remesh_cgal/remesh_fault_cgal.cpp`
- [created] `experimental_mesh_refinement/remesh_cgal/CMakeLists.txt`
- [created] `experimental_mesh_refinement/remesh_cgal/build/…` (CGAL binary)
- [created] `experimental_mesh_refinement/test_fault_triangle_quality.py`
- [created] `experimental_mesh_refinement/SAFS-…-ALT6_500m_clean_clip_nwcut_triq.stl`
- [created] `experimental_mesh_refinement/safs_fault_box_nwcut_500m_lcfar3000_z0embed_triq.msh`
- [modified] none (existing code/meshes untouched; existing `run_z0cut_meshing.py` / `check_mesh_quality.py` / `locate_fault_slivers.py` invoked by absolute path only)

1.6 Reproduce

```
# 1. build the CGAL tool (once)
CG=/Users/chunhuizhao/miniforge/envs/cgal-61
cd experimental_mesh_refinement/remesh_cgal
$CG/bin/cmake -S . -B build -DCMAKE_PREFIX_PATH=$CG -DCMAKE_CXX_COMPILER=$CG/bin/clang++
$CG/bin/cmake --build build -j

# 2. remesh the STL (pythonenv)
PY=/Users/chunhuizhao/miniforge/envs/pythonenv/bin/python
cd experimental_mesh_refinement
$PY remesh_fault_stl.py \
    ../meshing/results/stl_nwcut/SAFS-SAFZ-MULT-San_Andreas_fault_Fuis-ALT6_500m_clean_clip_nwcu
    SAFS-SAFZ-MULT-San_Andreas_fault_Fuis-ALT6_500m_clean_clip_nwcut_triq.stl

# 3. re-mesh with the EXISTING mesher (new output file)
$PY ../meshing/code/run_z0cut_meshing.py \
    --stl SAFS-SAFZ-MULT-San_Andreas_fault_Fuis-ALT6_500m_clean_clip_nwcut_triq.stl \
    --out safs_fault_box_nwcut_500m_lcfar3000_z0embed_triq.msh

# 4. verify
$PY ../meshing/code/check_mesh_quality.py safs_fault_box_nwcut_500m_lcfar3000_z0embed_triq.msh
$PY -m pytest -q test_fault_triangle_quality.py
```

1.7 Known limitations / not done

- **Scoped to the 500 m lcfar3000 mesh** (the named target). The tool generalizes (parameterized by STL + median edge), but the 1000 m / 250 m / zgraded variants were not regenerated.
- **Downstream velocity/stress projections** onto the new mesh were not re-run (separate follow-up once the `_triq` mesh is accepted).
- **CGAL binary is committed under `remesh_cgal/build/`** for convenience; rebuild it (step 1) if the `cgal-61` env changes.